

Transaction Concept

- A **transaction** is a unit of program execution that accesses and possibly updates various data items.
- E.g. transaction to transfer \$50 from account A to account B:
 1. `read(A)`
 2. `A := A - 50`
 3. `write(A)`
 4. `read(B)`
 5. `B := B + 50`
 6. `write(B)`
- Two main issues to deal with:
 - ✓ Failures of various kinds, such as hardware failures and system crashes
 - ✓ Concurrent execution of multiple transactions

A transaction is delimited by statements (or function calls) of the form **begin transaction** and **end transaction**. The transaction consists of all operations executed between the begin transaction and end transaction.

This collection of steps must appear to the user as a single, indivisible unit. Since a transaction is indivisible, it either executes in its entirety or not at all. Thus, if a transaction begins to execute but fails for whatever reason, any changes to the database that the transaction may have made must be undone. This requirement holds regardless of whether the transaction itself failed (e.g., if it divided by zero), the operating system crashed, or the computer itself stopped operating.

T_1	T_2
$R(A)$	
$R(B)$	
	$w(A)$
	$w(B)$

$R_1(A) \quad R_1(B) \quad w_2(A) \quad w_2(B)$

Example of Fund Transfer

- Transaction to transfer \$50 from account A to account B:

1. `read(A)`
2. `A := A - 50`
3. `write(A)`
4. `read(B)`
5. `B := B + 50`
6. `write(B)`

A: 100 B: 200
A: 50 B: 250

- Atomicity requirement**

- If the transaction fails after step 3 and before step 6, money will be "lost" leading to an inconsistent database state
 - Failure could be due to software or hardware
- The system should ensure that updates of a partially executed transaction are not reflected in the database

- Durability requirement** — once the user has been notified that the transaction has completed (i.e., the transfer of the \$50 has taken place), the updates to the database by the transaction must persist even if there are software or hardware failures.

The **Recovery Manager** is responsible for ensuring Atomicity and Durability.

The basic idea behind ensuring atomicity is this: The database system keeps track (on disk) of the old values of any data on which a transaction performs a write. This information is written to a file called the log. If the transaction does not complete its execution, the database system restores the old values from the log to make it appear as though the transaction never executed.

log

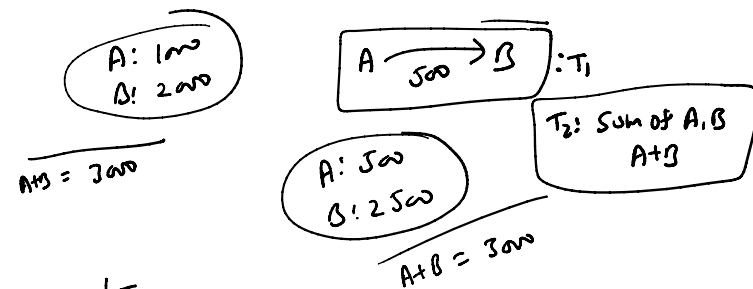
$$A_{old} = 100$$

$$A_{new} = 100 - 50 = 50$$

Consistency

Example of Fund Transfer (Cont.)

- Consistency requirement in above example:
 - The sum of A and B is unchanged by the execution of the transaction
- In general, consistency requirements include
 - Explicitly specified integrity constraints such as primary keys and foreign keys
 - Implicit integrity constraints
 - e.g., sum of balances of all accounts, minus sum of loan amounts must equal value of cash-in-hand
 - A transaction must see a consistent database.
 - During transaction execution the database may be temporarily inconsistent.
 - When the transaction completes successfully the database must be consistent



T ₁	T ₂
R(A)	
A = A - 500	
w(A)	
	R(A)
	2500
	R(B)
	A+B
	2500
	R(B)
	B = B + 500
	w(B)

500 : A
2000 : B

T₁ → T₂
T₂ → T₁

S ₁	T ₁	T ₂
R(A)		
w(A)		
		R(A)
	R(B)	
	w(B)	
		R(B)
		R(B)
		A+B = 3000

S ₂	T ₁	T ₂
R(A)		
w(A)		
		R(A)
	R(B)	
	w(B)	
		R(B)
		R(A)
		R(B)
		A+B = 3000

T₁ → T₂

S₁ ≡ S₂

Isolation

Example of Fund Transfer (Cont.)

- **Isolation requirement** — if between steps 3 and 6, another transaction T2 is allowed to access the partially updated database, it will see an inconsistent database (the sum $A + B$ will be less than it should be).

T1

1. **read**(A)
2. $A := A - 50$
3. **write**(A)
4. **read**(B)
5. $B := B + 50$
6. **write**(B)

T2

read(A), read(B), print(A+B)

- Isolation can be ensured trivially by running transactions **serially**
 - That is, one after the other.
- However, executing multiple transactions concurrently has significant benefits, as we will see later.

Ensuring the isolation property is the responsibility of a component of the database system called the **concurrency-control system**.

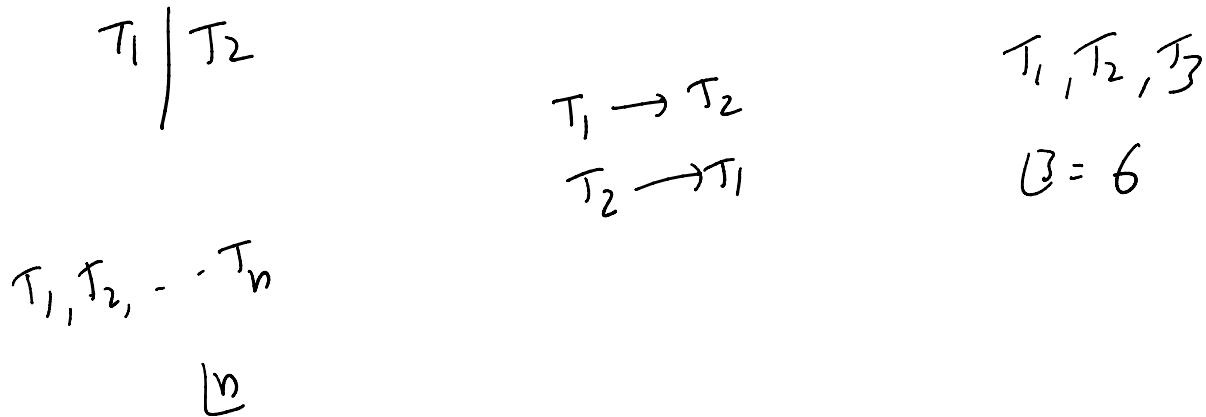
Schedules

- **Schedule** – a sequences of instructions that specify the chronological order in which instructions of concurrent transactions are executed
 - A schedule for a set of transactions must consist of all instructions of those transactions
 - Must preserve the order in which the instructions appear in each individual transaction.
- A transaction that successfully completes its execution will have a commit instructions as the last statement
 - By default transaction assumed to execute commit instruction as its last step
- A transaction that fails to successfully complete its execution will have an abort instruction as the last statement

Serial schedule and Concurrent schedule

Serial schedule

Schedules in which the transactions are executed non-interleaved, i.e., a serial schedule is one in which no transaction starts until a running transaction has ended are called serial schedules.



Concurrent schedule

Schedules in which the transactions are executed interleaved.
Simultaneous execution of two or more transactions.

Concurrent Executions

- Multiple transactions are allowed to run concurrently in the system.
Advantages are:
 - Increased processor and disk utilization**, leading to better transaction throughput
 - e.g., one transaction can be using the CPU while another is reading from or writing to the disk
 - Reduced average response time** for transactions: short transactions need not wait behind long ones.
- Concurrency control schemes** – mechanisms to achieve isolation
 - That is, to control the interaction among the concurrent transactions in order to prevent them from destroying the consistency of the database

ACID Properties

A **transaction** is a unit of program execution that accesses and possibly updates various data items. To preserve the integrity of data the database system must ensure:

- **Atomicity.** Either all operations of the transaction are properly reflected in the database or none are.
- **Consistency.** Execution of a transaction in isolation preserves the consistency of the database.
- **Isolation.** Although multiple transactions may execute concurrently, each transaction must be unaware of other concurrently executing transactions. Intermediate transaction results must be hidden from other concurrently executed transactions.
 - That is, for every pair of transactions T_i and T_j , it appears to T_i that either T_j finished execution before T_i started, or T_j started execution after T_i finished.
- **Durability.** After a transaction completes successfully, the changes it has made to the database persist, even if there are system failures.

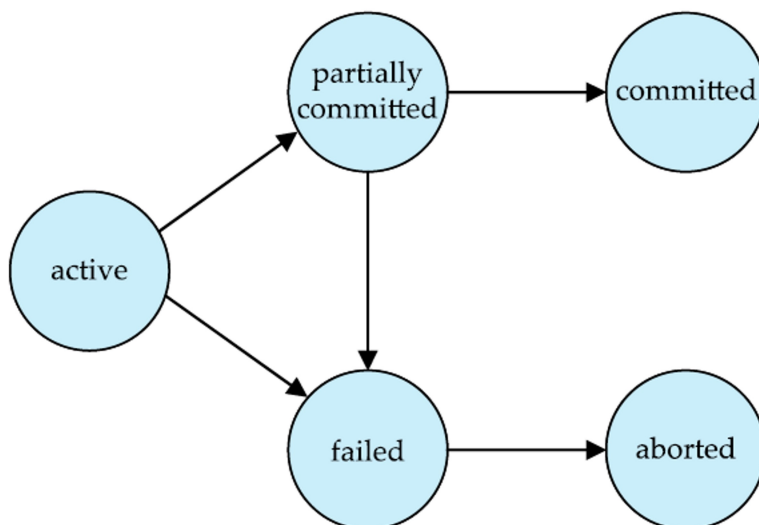
The acronym ACID is sometimes used to refer to the four properties of transactions that we have presented here: atomicity, consistency, isolation and durability.

There are four important properties of transactions that a DBMS must ensure to maintain data in the face of concurrent access and system failures:

1. Users should be able to regard the execution of each transaction as **atomic**: either all actions are carried out or none are. Users should not have to worry about the effect of incomplete transactions (say, when a system crash occurs).
2. Each transaction, run by itself with no concurrent execution of other transactions, must preserve the consistency of the database. This property is called **consistency**, and the DBMS assumes that it holds for each transaction. Ensuring this property of a transaction is the responsibility of the user.
3. Users should be able to understand a transaction without considering the effect of other concurrently executing transactions, even if the DBMS interleaves the actions of several transactions for performance reasons. This property is sometimes referred to as **isolation**: Transactions are isolated, or protected, from the effects of concurrently scheduling other transactions.
4. Once the DBMS informs the user that a transaction has been successfully completed, its effects should persist even if the system crashes before all its changes are reflected on disk. This property is called **durability**.

Transaction State

- **Active** – the initial state; the transaction stays in this state while it is executing
- **Partially committed** – after the final statement has been executed.
- **Failed** -- after the discovery that normal execution can no longer proceed.
- **Aborted** – after the transaction has been rolled back and the database restored to its state prior to the start of the transaction. Two options after it has been aborted:
 - restart the transaction
 - can be done only if no internal logical error
 - kill the transaction
- **Committed** – after successful completion.



CONCURRENT EXECUTION OF TRANSACTIONS

The DBMS interleaves the actions of different transactions to improve performance, in terms of increased throughput or improved response times for short transactions, but not all interleavings should be allowed.

Some Anomalies Associated with Interleaved Execution

We now illustrate three main ways in which a schedule involving two consistency preserving, committed transactions could run against a consistent database and leave it in an inconsistent state. Two actions on the same data object **conflict** if at least one of them is a write. The three anomalous situations can be described in terms of when the actions of two transactions $T1$ and $T2$ conflict with each other: in a **write-read (WR) conflict** $T2$ reads a data object previously written by $T1$; we define **read-write (RW)** and **write-write (WW)** conflicts similarly.

Reading Uncommitted Data (WR Conflicts)

The first source of anomalies is that a transaction $T2$ could read a database object A that has been modified by another transaction $T1$, which has not yet committed. Such a read is called a **dirty read**. A simple example illustrates how such a schedule could lead to an inconsistent database state. Consider two transactions $T1$ and $T2$, each of which, run alone, preserves database consistency: $T1$ transfers \$100 from A to B , and $T2$ increments both A and B by 6 percent (e.g., annual interest is deposited into these two accounts). Suppose that their actions are interleaved so that (1) the account transfer program $T1$ deducts \$100 from account A , then (2) the interest deposit program $T2$ reads the current values of accounts A and B and adds 6 percent interest to each, and then (3) the account transfer program credits \$100 to account B . The corresponding schedule, which is the view the DBMS has of this series of events, is illustrated in Figure 18.2. The result of this schedule is different from any result that

$T1$	$T2$
$R(A)$	
$W(A)$	
	$R(A)$
	$W(A)$
	$R(B)$
	$W(B)$
	Commit
$R(B)$	
$W(B)$	
Commit	

Figure 18.2 Reading Uncommitted Data

we would get by running one of the two transactions first and then the other. The problem can be traced to the fact that the value of A written by $T1$ is read by $T2$ before $T1$ has completed all its changes.

The general problem illustrated here is that $T1$ may write some value into A that makes the database inconsistent. As long as $T1$ overwrites this value with a ‘correct’ value of A before committing, no harm is done if $T1$ and $T2$ run in some serial order, because $T2$ would then not see the (temporary) inconsistency. On the other hand, interleaved execution can expose this inconsistency and lead to an inconsistent final database state.

Note that although a transaction must leave a database in a consistent state *after* it completes, it is not required to keep the database consistent while it is still in progress. Such a requirement would be too restrictive: To transfer money from one account

to another, a transaction *must* debit one account, temporarily leaving the database inconsistent, and then credit the second account, restoring consistency again.

Unrepeatable Reads (RW Conflicts)

The second way in which anomalous behavior could result is that a transaction $T2$ could change the value of an object A that has been read by a transaction $T1$, while $T1$ is still in progress. This situation causes two problems.

First, if $T1$ tries to read the value of A again, it will get a different result, even though it has not modified A in the meantime. This situation could not arise in a serial execution of two transactions; it is called an **unrepeatable read**.

Second, suppose that both $T1$ and $T2$ read the same value of A , say, 5, and then $T1$, which wants to increment A by 1, changes it to 6, and $T2$, which wants to decrement A by 1, decrements the value that it read (i.e., 5) and changes A to 4. Running these transactions in any serial order should leave A with a final value of 5; thus, the interleaved execution leads to an inconsistent state. The underlying problem here is that although $T2$'s change is not directly read by $T1$, it invalidates $T1$'s assumption about the value of A , which is the basis for some of $T1$'s subsequent actions.

Overwriting Uncommitted Data (WW Conflicts)

The third source of anomalous behavior is that a transaction $T2$ could overwrite the value of an object A , which has already been modified by a transaction $T1$, while $T1$ is still in progress. Even if $T2$ does not read the value of A written by $T1$, a potential problem exists as the following example illustrates.

Suppose that Harry and Larry are two employees, and their salaries must be kept equal. Transaction $T1$ sets their salaries to \$1,000 and transaction $T2$ sets their salaries to \$2,000. If we execute these in the serial order $T1$ followed by $T2$, both receive the salary \$2,000; the serial order $T2$ followed by $T1$ gives each the salary \$1,000. Either of these is acceptable from a consistency standpoint (although Harry and Larry may prefer a higher salary!). Notice that neither transaction reads a salary value before writing it—such a write is called a **blind write**, for obvious reasons.

Now, consider the following interleaving of the actions of $T1$ and $T2$: $T1$ sets Harry's salary to \$1,000, $T2$ sets Larry's salary to \$2,000, $T1$ sets Larry's salary to \$1,000, and finally $T2$ sets Harry's salary to \$2,000. The result is not identical to the result of either of the two possible serial executions, and the interleaved schedule is therefore not serializable. It violates the desired consistency criterion that the two salaries must be equal.

R w
w R
w w

T ₁	T ₂
w(B)	R(A)

T ₁	T ₂
w(A)	
	R(A)

T ₁	T ₂
	R(A)
w(A)	

wrong value

T ₁	T ₂
R(A)	
	w(A)

T ₁	T ₂
	w(A)
R(A)	

T ₁	T ₂
w(A) w(B)	
	w(A) w(B)

A, B = 2000

T₁: A, B to low

T₂: A, B to 2000

A and B values must be equal

T ₁	T ₂
	w(A) w(B)
w(A) w(B)	

A, B = low

T ₁	T ₂
w(A)	
	w(A) w(B)
w(B)	

A = ~~low~~ 2000

B = ~~2000~~ low

A 2000
B low